

Yayın Merkezi — Teknik Referans Dokümanı

Bu doküman, projenin tüm backend (Quarkus/Java) paketlerini ve frontend (React/TypeScript) katmanlarını, kod içi Türkçe yorumlardan çıkarılan tasarım gerekçeleriyle birlikte anlatır. Otomatik olarak, kaynak kodun doğrudan taranmasıyla üretilmiştir.

1. Mimari Genel Bakış

```

kaynak (RTSP/RTMP/HLS/Icecast) → MediaMTX → tarayıcı (HLS)
|
├── video-worker : DVR kaydı, VAD, klip/video işleme, kanal önizleme
└── backend      : REST API, WebSocket, kuyruk yönetimi, admin panel

```

Aynı derlenmiş jar iki ayrı Docker imajında çalışır (Dockerfile.jvm → backend, Dockerfile.worker → video-worker). Hangi bileşenin hangi konteynerde aktif olacağı ortam değişkeni bayraklarıyla belirlenir (dvr.recorder-enabled, vad.enabled, clips.worker.enabled, videos.worker.enabled, channels.snapshot-enabled vb.) — CDI bean'leri her iki konteynerde de örneklenir, bayrak unutulursa aynı iş iki kez yapılır. Ayrım nedeni: video-worker'da ffmpeg + VAAPI/NVENC donanım erişimi var, backend'de yalnızca -c copy işlemleri için düz ffmpeg var.

Konteynerler arası tek bağ Redis ve Postgres'tir — doğrudan servis çağrısı yoktur. Doğruluk kaynağı her zaman veritabanı; Redis yalnızca bildirim/hızlandırma taşıyıcı, kaybolursa ilgili modülün kendi yoklama (polling) döngüsü durumu toparlar. Bu desen üç ayrı yerde tekrarlanır: DVR sinyalleşmesi (DvrSignal), klip/video kuyrukları (ClipQueue / VideoQueue) ve canlı altyazı dağıtımı (SubtitleBroadcaster).

docker-compose servisleri: postgres, keycloak + keycloak-postgres, minio, redis, mediamtx, backend, triton, video-worker, frontend, ve izleme yığını: prometheus, grafana, loki, promtail, dcgm-exporter, postgres-exporter, keycloak-postgres-exporter, redis-exporter.

Teknoloji yığını: Quarkus + Panache (Hibernate ORM) + Flyway (V1→V30), Keycloak OIDC (kimlik), MinIO (nesne depolama), Redis (kuyruk/pub-sub), Prometheus+Grafana+Loki+Promtail (izleme), Triton Inference Server (STT/çeviri), React 19 + Vite + TypeScript + Tailwind v4 + shadcn/ui (frontend).

2. Kimlik Doğrulama ve Kullanıcı Yönetimi

2.1 org.example.auth

Bu paket, kimlik doğrulamanın (giriş, token yenileme, çıkış, şifre doğrulama) Keycloak ile köprüsünü kurar. Uygulama hiçbir zaman şifre saklamaz veya karşılaştırmaz; sadece Keycloak'a delege eder ve gelen sonucu güvenli bir şekilde uygulamaya özel hata mesajlarına çevirir.

AuthResource.java

- /api/auth altında login, refresh, logout uçlarını sunan REST kaynağı.
- Üç uç da @PermitAll — bilinçli bir tasarım kararı: bunlar kimlik doğrulamanın kendisini yaptığı için önceden bir token isteyemez.
- logout ucunun da açık (token'sız) olması özellikle vurgulanmış: access token süresi dolduktan sonra kullanıcı çıkış yapamayıp refresh token Keycloak'ta geçerli kalmaya devam ederdi. Güvenlik zaten refresh token'ın kendisine dayanıyor.
- Tüm iş mantığını AuthService'e devreder.

AuthService.java

- Giriş/yenileme/çıkış akışının iş mantığı; KeycloakTokenClient REST client'ını sarmalar.
- login: password grant ile token alır, başarı/başarısızlığı EtkinlikService.kaydetGirisDenemesi ile denetim izine (GIRIS / GIRIS_BASARISIZ) yazar.
- Kritik güvenlik kararı: loginFailure metodu, kullanıcı adının var olup olmadığı, hesabın kapalı mı yoksa şifrenin mi yanlış olduğu ayrımını dışarı hiç vermez — genel "Kullanıcı adı veya şifre hatalı" mesajı döner (kullanıcı adı numaralandırma saldırısına karşı). Tek istisna: Keycloak'ın Account is not fully set up hatası (geçici şifreyle açılan hesap) ayrı ve açıklayıcı bir mesajla iletiliyor.
- refresh: 400/401 durumunda "oturum süresi dolmuş, yeniden giriş yapın" mesajı.
- logout: Çıkış öncesi denetim kaydı atarken kullanıcı kimliğini JWT imzası doğrulanmadan okur — bu bilinçli, güvenlik kararı buna dayanmıyor, sadece "kim çıkış yaptı" alanı denetim izi içindir.
- verifyPassword: Kullanıcının kendi şifresini değiştirmeden önce mevcut şifreyi doğrulamak için kullanılır. Keycloak Admin API'de "bu şifre doğru mu" diye soran bir uç olmadığı için, doğrulama fiilen bir giriş denemesi (passwordGrant) yapılarak simüle ediliyor.

KeycloakTokenClient.java

- Keycloak'un OIDC token uçlarına (/realms/{realm}/protocol/openid-connect/token ve /logout) MicroProfile REST Client ile erişen arayüz. Çalışması için Keycloak client'ında Direct access grants özelliğinin açık olması gerekiyor.

dto paketi

- LoginRequest / LogoutRequest / RefreshRequest: basit record DTO'lar. LoginRequest'te şifre alanı uzunluk doğrulamasına tabi değil — burada bir kayıt değil, var olan bir kimlik bilgisinin sınanması söz konusu.
- TokenResponse: Keycloak'un OAuth2 token yanıtını eşler, @JsonIgnoreProperties(ignoreUnknown = true) ile kullanılmayan alanlar atlanır.

2.2 org.example.user

Kullanıcı yönetimini (Keycloak Admin API ile CRUD, rol atama, şifre işlemleri) ve Keycloak ↔ yerel veritabanı senkronizasyonunu kapsar. **Doğruluk kaynağı her zaman Keycloak'tır**; yerel `users` tablosu sadece uygulama verisinin bir kullanıcıya foreign key ile bağlanabilmesi için tutulan bir aynadır.

AdminUserResource.java

- `/api/admin/users` — yönetici kullanıcı yönetimi. Sınıf düzeyinde `@RolesAllowed(Roles.YONETICI)` tüm uçları kapsar.
- Yol parametresi her zaman **Keycloak kullanıcı id'si**; yerel `users.id` dışarıya hiç açılmaz.

CurrentUserResource.java

- `/api/users/me` — kullanıcının kendi hesabı. Hedef kullanıcı her zaman token'ın sahibi; istekten kullanıcı id'si asla alınmıyor — IDOR riski yapısal olarak imkânsız.
- `quota()` : klip/kayıt/ekran görüntüsü/video toplam depolama kullanımını döner.

UserService.java

- Kullanıcı yönetiminin tek giriş noktası: **okuma Keycloak'tan, yazmada önce Keycloak, sonra yerel `users` aynalanır**.
- `changeRole` : Kritik koruma — bir yönetici kendi rolünü YONETICI dışına düşüremez (aksi halde tek yönetici kendini izleyiciye çevirip paneli kilitleyebilir).
- `delete` : Kendi hesabını silmeyi engeller.
- `sync()` : Keycloak → yerel `users` tek yönlü aynalama; Keycloak'ta artık bulunmayanları **silmez, sadece raporlar (orphaned)** — uygulama verisi bu satırlara FK ile bağlı olabilir.
- `clientId()` : Roller **client rolü** (realm rolü değil) olarak tanımlı; client rolü uçları clientId yerine Keycloak'un iç UUID'sini ister.

UserProvisioningService.java / UserProvisioningFilter.java

- Keycloak kullanıcısının yerel `users` satırını **talep anında (just-in-time)** oluşturur/günceller — yönetici Keycloak konsolundan doğrudan kullanıcı açabildiği için.
- `UserProvisioningFilter` : `@Priority(Priorities.AUTHORIZATION + 100)` ile her kimlik doğrulanmış istekte tetiklenir; merkezi olmasının nedeni her uç kendi kontrol etseydi birinin unutulmasında sessizce bozulacak olması.

Roles.java

- Rol sabitleri: `YONETICI` , `MODERATOR` (değeri `"Moderatör"`) , `IZLEYICI` . Bu sabitler `roles.name` , Keycloak realm/client rolü adı ile **üç yerde birebir aynı** olmak zorunda.

TokenClaims.java

- `preferred_username` claim'i açıkça okunuyor çünkü Quarkus'un varsayılan principal ismi realm ayarına göre UUID dönebiliyor.

entity / dto

- `AppUser` : şifre burada tutulmaz; `keycloakId` bağ noktasıdır.
- `KullaniciSayfasiDto(items, total, first, max)` : gerçek toplam sayı Keycloak'ın dedike `count()` ucundan geliyor (per-user sorgu değil).
- `ResetPasswordRequest` : mevcut şifre istenmez (yetki admin rolünden geliyor); `ChangePasswordRequest` : mevcut şifre zorunlu.

2.3 org.example.viewer

ViewerPresence.java

- Kanal/radyo başına, **sekme (tabId) bazlı** eşzamanlı izleyici/dinleyici sayacı, bellekte tutulur.
- Kritik gerekçe**: MediaMTX'in kendi "reader" sayısı doğruluk kaynağı olamaz — kalite değişimi, ağ kesintisi sonrası yeniden bağlanma veya hls.js'in paralel segment istekleri birden fazla reader gibi görünmesine yol açıyor (gözlemlenmiş: "tek oturum açık ama üç izleyici görünüyor").
- `nabiz()` : heartbeat kaydeder; `supur()` (`@Scheduled every=10s`) : TTL=40 sn (15 sn'lik heartbeat periyoduna göre 2-3 kaçırılan sinyale tolerans).

2.4 Veritabanı ve Config (Kimlik)

Migration	İçerik
V1	<code>roles</code> , <code>users</code> tabloları; roller <code>ADMIN / MODERATOR / VIEWER</code> seed edilir.
V2	Rol isimleri Türkçeleştirilir, ASCII tutulur (<code>YONETICI</code> , <code>IZLEYICI</code>) — Keycloak/JWT/URL'de aksan sorunu çıkmasın diye.
V3	Nihai aksanlı Türkçe hale çevrilir (<code>Yönetici</code> , <code>Moderatör</code> , <code>İzleyici</code>); <code>role_id</code> FK olduğu için id'ler değişmez.

Önemli config: `quarkus.oidc.auth-server-url` , `quarkus.oidc.client-id/credentials.secret` ; **role-claim-path bilinçli olarak boş bırakılıyor** — roller `resource_access.<client>.roles` altında (client rolü), açık yol yazmak client adını ikinci kez gömer ve isim değişince yetkilendirme sessizce bozulur. `quarkus.keycloak.admin-client.*` (CLIENT_CREDENTIALS grant, `manage-users / view-users / query-users` rolleri gerekli).

3. Kanal, Medya ve Oynatma

3.1 org.example.channel

Kanalların CRUD yaşam döngüsünü, MediaMTX ile senkronizasyonunu, çözünürlük merdivenini (rendition ladder), kaynak doğrulamayı, önizleme karesi yakalamayı ve kanal silmeyi yönetir. Paketin kalbi "veritabanı kalıcı gerçek, MediaMTX çalışan durum" ayrımı — her yazma önce DB'ye sonra MediaMTX'e gider.

Channel.java (entity)

- `channels` tablosu. `resolvedSourceUrl` bilinçli olarak `sourceUrl`'i **ezmiyor**: kullanıcı kendi girdisini görmeye devam etsin.
- `dvrEnabled` bilerek `active`'den ayrı bir bayrak: 6 Mbps'lik kanal 7 günde ~454 GB yazdığı için kayıt, yayında olmaktan bağımsız bilinçli bir karar.

ChannelResource.java

- `/api/channels`. Okuma `@Authenticated`, yazma `@RolesAllowed({YONETICI, MODERATOR})`.
- `PUT/DELETE /{id}/izleyici/{tabId}`: sekme bazlı izleme nabızı (`ViewerPresence`).
- `POST /{id}/silme DELETE değil`: şifre gövdede taşınıyor, sorgu parametresinde gitseydi erişim günlüklerine düz metin düşerdi. Kullanıcı adı token'dan alınır.

ChannelService.java

- Kritik kural**: her yazma DB + MediaMTX'i birlikte değiştirir, sıra her zaman önce DB sonra MediaMTX.
- `capacity()/requireCapacity()`: `channels.max-active` sınırını **uygulama tarafında** açık hata olarak uygular çünkü MediaMTX'in kendisi sınırı uygulamıyor ve aşıldığında **tüm** kanallarda bozulmaya yol açıyor.
- `restoreActiveChannels()`: hata tek tek yutulur — bir kanalın kaynağı erişilemezken diğerlerinin ayağa kalkmaması saçma olurdu.

MediaMtxClient.java / MediaMtxService.java

- MediaMTX v3 REST API'si. `/config/paths/...` (yapılandırma) vs `/paths/...` (çalışma zamanı durumu) ayrımı önemli — path'in tanımlı olması yayının aktığı anlamına gelmiyor.
- `applyPath()`:
 - Kayıt her zaman KAYNAK path'ine** yazılır, bir rendition'a değil — önceden bir rendition'a yazılıyordu, iki bedeli vardı: kalite kaybı ve **sessiz boş kayıt** (o rendition'ı üreten ffmpeg çalışmıyorsa kayıt içine hiçbir şey yazmıyordu, gerçekten yaşanmış).
 - Rendition'lar talebe bağlı**: her biri kendi `runOnDemand` kancasıyla ilk okuyucu gelince başlıyor, `channels.rendition-close-after` sonra kapanıyor.
- `pathStates()`: MediaMTX'e ulaşılamazsa **hata fırlatmaz, boş harita döner** — kanal listesini tamamen kullanılamaz hale getirmemeli.

ChannelDeletionService.java

- Kanal silme akışı: özet çıkarma, şifre doğrulama, üç ayrı kovadan (klip/ekran görüntüsü/DVR) nesne silme.
- Klip/ekran görüntüsü/DVR **ayrı ayrı** seçilebiliyor: klip emek harcanmış çıktı, ekran görüntüsü tek tıkla yeniden alınabilir, DVR büyük ve geçici.
- DVR farklı davranıyor**: `dvr_segments` her koşulda CASCADE gider; seçilebilen tek şey MinIO baytlarının hemen mi silineceği yoksa saklama kuralına mı bırakılacağı.
- Sıra kritik: **önce şifre**, sonra nesneler, en son satırlar.
- Silme parça parça (`BATCH=500`) ve **transaction dışında** — 7 günlük DVR'da kanal başına ~20 bin segment olabiliyor.

ChannelSnapshotStorage.java / ChannelSnapshotWorker.java

- Kanal önizleme kareleri: her kanal için **tek nesne**, her yakalamada üzerine yazılır — arşiv değil "şu an nasıl görünüyor" göstergesi.
- Yakalama **doğrudan RTSP** kullanır, HLS değil — MediaMTX'in isteğe bağlı HLS muxer'ını tetiklemek istenmiyor. Yalnızca video-worker'da çalışır (ffmpeg gerektirir).

HlsPlaylist.java / SourceProbe.java

- `SourceProbe`, iki ölçülmüş sorunu çözer:
 - Master playlist tuzağı**: MediaMTX master playlist verildiğinde en yüksek bant genişlikli varyantı seçiyor; segmentler gohlslib'in ~4 MB sınırını aşarsa yayın **hiç başlamıyor** (`max recorded size exceeded`, kullanıcı hiçbir hata görmüyor). Ölçüm: TRT 720p (3.01 MB/segment) çalışıyor, 1080p (4.29 MB) düşüyor.
 - Merdivenin kaynağın üstüne çıkabilmesi.
- Belirleyici bant genişliği değil **segment başına bayt**.
- `probe()` **hata fırlatmaz** — kaynak erişilemiyorsa kanal yine kaydedilebilmeli.

TranscodeCommand.java

- Çözünürlük düşürme için MediaMTX `runOnDemand` kancasına ffmpeg komutu üretir. **Rendition başına bağımsız süreç** modeli — kimse izlemiyorsa süreç hiç çalışmıyor, asıl kazanç bu.

dto özet

- `ChannelDto.hlsUrl` bilinçli olarak **görelî** — mutlak olsaydı arayüze girilen adresle yayının geldiği adres ayrışabilirdi (gerçek olay: LAN IP'den açılan arayüz alan adına ayarlı mutlak adresten yayın isteyip bağlanamamış).
- `viewers` artık `ViewerPresence` sekme heartbeat'inden, MediaMTX reader sayısından değil.

3.2 org.example.media

VideoEncoder.java

- Üç değer: `NVENC` (üretim hedefi), `VAAPI` (Intel/AMD, geliştirme makinesinde kullanılan seçenek), `YAZILIM` (`libx264`, donanım yoksa).

- **Ölçülen maliyet:** 1080p kaynak, rendition başına VA-API ~%14 CPU, yazılım ~%142 — 16 kanalda fark 2,2 çekirdek ile 22,7 çekirdek arasında. MediaMTX'in kendisi 12 kanal + 15 radyoyu %25 CPU ile taşıyor — **ölçeklemenin önündeki duvar kanal sayısı değil, transcode.**
- `-map 0:v:0 -map 0:a:0` açıkça seçiliyor — seçilmezse filtre yanlış akışa uygulanıp düşüyor (ölçümde yaşandı).
- Kare yakalamada yalnızca **çözme** hızlandırılıyor — `hwaccel_output_format` verilmiyor, verilseydi kare GPU'da kalıp yazılım `mjpeg` kodlayıcıya ulaşamazdı.

3.3 org.example.playback

PlaybackHealthMetrics.java

- Oynatma hatası/takılma telemetrisi; `oynatma_hata_toplam` / `oynatma_takilma_toplam` Micrometer counter'ları.

3.4 Veritabanı ve Config (Kanal)

Migration	İçerik
V4	<code>channels</code> tablosu; kısmi indeks <code>idx_channels_active where active</code> .
V6	<code>renditions</code> sütunu — global değil kanal bazında (ölçüm: 2.29 Mbps kaynağa 2500k'lık 720p uygulanınca çıktı 2.51 Mbps oldu, saf israf).
V12	<code>resolved_source_url</code> , <code>source_width/height</code> — master playlist tuzağı çözümü.
V21	<code>clips / screenshots.channel_id</code> CASCADE → SET NULL + <code>channel_name</code> kopya sütunu; <code>dvr_segments</code> kasıtlı CASCADE kalır.

Önemli config: `channels.max-active` (16), `channels.encoder` (NVENC/VA-API/YAZILIM), `channels.hls-max-segment-bytes` (3.5 MB — bilinen çalışan/düşen arası), `channels.snapshot-*`, `mediamtx.hls-base-url` (görelili).

4. Canlı Altyazı (VAD → Triton → Yayın)

4.1 Boru hattı

```
ffmpeg(RTSP, ses-only) → 512-örneklik kareler → Silero VAD v5
→ SpeechSegmenter (durum makinesi) → SpeechSegment (PCM + mutlak zaman)
→ kanal başına kuyruk (4) → round-robin gönderici havuzu
→ Triton HTTP: whisper (task=translate) → İngilizce pivot metin
→ Triton HTTP: N × marian_en_<dil> (paralel) → tr/de/ru...
→ SubtitleService.kaydetVeyaBirlestir() → Postgres `altyazilar`
→ Redis PUBLISH altyazi:<channelId>
→ (backend) Redis PSUBSCRIBE altyazi:* → SubtitleBroadcaster → WebSocket
→ tarayıcı: mutlak zaman damgasıyla playingDate() karşısında eşleştir
```

4.2 Ses alma — AudioStream.java

```
ffmpeg -v error -rtsp_transport tcp -allowed_media_types audio \
-i rtsp://mediamtx:8554/<path> \
-vn -ac 1 -ar 16000 -f s16le -
```

- `-allowed_media_types audio`: video track'i RTSP SETUP aşamasında hiç istenmiyor — **ölçülmüş:** CPU %1,5 → %0,8.
- Zaman **duvar saat** ile değil **örnek sayacı**yla tutuluyor (`anchor + samplesRead×1000/16000 ms`) — ağ tıkanmasında zaman kaymasın diye.
- stderr ayrı thread'de boşaltılıyor — okunmazsa boru dolar, ffmpeg sessizce donar.

4.3 VAD — SileroVad.java + SpeechSegmenter.java

Model: Silero v5 (v4 değil — `requireV5Signature()` ile zorunlu). v4'te yanlış pencere boyutu hata vermeden farklı sonuç üretiyordu; v5 yalnızca 512 örneği kabul ediyor.

Kare = 512 örnek @16kHz = 32 ms. Her karede önceki karenin son 64 örneği (context) + LSTM durumu (state `[2, 1, 128]`) veriliyor. Context atlanırsa model **hata vermeden** her şeyi sessizlik sanıyor — ölçülmüş: bağlamsız çağrıda konuşma oranı %0, bağlamla %97.

SpeechSegmenter üç durumlu makine:

```
KAPALI —(N kare p>0.50)→ AÇIK —(600ms p<0.35)→ BEKLEYEN
|                                     |
|                                     |
| (p>0.50 devam) —————— |
| (süre ≥ MIN_EMIT_MS, yayınla) —————— |
```

- Açma/kapama eşiği kasıtlı farklı (0.50/0.35) — histerezis, sınırda gezinen skorun bölütü saniyede birkaç kez açıp kapatmasını önler.
- **BEKLEYEN** durumu kısa parçaları birleştirir; zaman damgaları düzülür kalır.
- **Geriye dönük tampon (preroll 500ms):** ilk hece kırılmasın diye.
- **MAX_SEGMENT_MS** (varsayılan 6000ms) — **gecikmenin ana belirleyicisi.** Ölçülen: 25 sn'lik pencerede üretim gecikmesi 22,5 sn, bunun 14,5 sn'si sadece bölütün kapanmasını beklemektir. Şimdi zorla kesiliyor (800ms örtüşmeyle).

4.4 Kanal kuyruklaması — VadService.java

- Kare döngüsü asla STT beklemez. **Kanal başına ayrı kuyruk** (kapasite 4): tek paylaşımlı kuyrukta konuşkan kanal kapasiteyi tüketip sessiz kanalların bölütlerini reddediyordu (head-of-line blocking).
- Dolunca en ESKİ atılır**, en yeni girer — eskiden tersiydi.
- Kuyruğa girmeden ÖNCE yaş kontrolü** (`altyazi.butce-ms` , 8000ms): bölüt bütçeyi çoktan aşmışsa hiç kuyruğa alınmıyor.
- round-robin dağıtım**: bir kanal işlenirken kuyruğunda başka bölüt kaldıysa sıranın sonuna yeniden eklenir.

4.5 Triton'a gönderim — TritonClient.java + whisper/model.py

Eskiden ayrı bir Python `stt-worker` mikroservisiydi; artık **doğrudan Triton Inference Server'a** KServe v2 HTTP protokolüyle konuşuluyor.

- PCM ham baytları JSON'un arkasına düz ekleniyor (binary-data-extension) — base64'ün %33 şişmesini önlemek için.
- Whisper `task=translate` ile çağrılıyor: kaynak dil ne olursa olsun tek geçişte İngilizce pivot metin üretiliyor. Triton'da `faster-whisper` + `BatchedInferencePipeline` ; batching Triton'ın kendi `dynamic_batching` 'i.
- "Anında yayınla" deseni**: pivot metin hazır olur olmaz hemen yayınlanır, sonra hedef dillere paralel çeviri isteği atılır. Bir dilin çevirisi geç kalırsa diğerleri beklemez.

4.6 Çeviri — marian_model.py (CTranslate2)

Üç geçiş yaşanmış, sonuncusunda ONNX Runtime'dan **CTranslate2**'ye taşınmış: ONNX Runtime'ın CUDA "caching allocator"ı değişken cümle uzunluğu yüzünden her yeni tensör şekli için yeni bellek bloğu açıp **asla geri vermiyordu** — gerçek 15 kanal yükünde 1 saatte 590MB'tan 5,1GB'a çıkmış.

- Metin cümle sınırından bölünüyor (Marian cümle düzeyinde eğitilmiş, uzun paragraf sessizce kırılıyor).
- EOS token'i elle ekleniyor** — CTranslate2'nin Marian config'i `add_source_eos=false` , eklenmezse encoder çıkışı bozulup model sonsuz tekrara giriyor (gerçek teste: "Das Wetter ist Wetter ist Wetter...").
- Tam dinamik dil (17 Ağustos)**: `export_models.py` , `STT_TARGET_LANGS` 'taki her dil için `templates/arian_model.py` şablonunu olduğu gibi kopyalar; model.py hiçbir dile özel kod içermiyor. Yeni dil eklemek yalnızca `.env` değişikliği.
- Varsayılan kalıp yok**: `MARIAN_MODELS` her dil için zorunlu, eksikse build açıkça hata verip durur — tahmin edilen bir repo adının 401 ile build ortasında patlamasındansa.
- `_DtypeDüzeltilmisConverter` : ctranslate2 4.8.1 + transformers 4.48.0 uyumsuzluğu (`dtype` kwarg'i) — küçük bir alt sınıfla düzeltilmiş.
- `fp16` export varsayılan — VRAM'i yarıya indirdi, kalite etkisi **ölçülmedi**; sorun çıkarsa `.env` 'de tek satırla `fp32` 'ye dönülebilir.

4.7 Kayıt — SubtitleService.java / Subtitle entity

Tablo: `altyazilar` (`channel_id` , `baslangic` , `bitis` , `kaynak_dil` , `guven` , `metinler` JSONB , `kesik`). Pivot ve her çeviri **ayrı ayrı** geldiği için aynı (`channelId` , `baslangic`) için birden fazla kez çağrı gelir: ilk çağrı satırı oluşturur, sonrakiler `metinler` haritasına **eklenir**, üzerine yazılmaz.

4.8 Dağıtım — Redis pub/sub, WebSocket

`video-worker` (üretici) ile `backend` (WebSocket) ayrı konteynerler. Yaşanmış hata: abonelik açılması **41 saniye** bloke kalmış (`No handler waiting for message`). İki sebep birlikte:

- Pub/sub, klip kuyruğunun `BLMOVE` 'uyla **aynı Redis istemcisini** paylaşıyordu → ayrı istemci (`quarkus.redis.pubsub.*`).
- Her izleyici gidişinde abonelik bırakılıp yeniden açılıyordu → artık **desenle tek abonelik** (`altyazi:*`), süreç boyunca açık.

`SubtitleSocket` : `/ws/altyazi/{channelId}` , kimlik doğrulaması yok (bilinen eksik). `SubtitleOverlay` açılışta REST ile son 60 sn'yi doldurur, sonrası WebSocket.

4.9 Ekranda gösterme — SubtitleOverlay.tsx — kritik nokta

```
baslangic <= playingDate() < bitis
```

`playingDate()` mesajın geldiği an değil, hls.js'in `EXT-X-PROGRAM-DATE-TIME` 'dan okuduğu karenin gerçek yayın anı. **Geç kalan bir bölüt geç gösterilmiyor, hiç gösterilmiyor** — oynatma kafası `bitis` 'i geçtiyse süzgeçten düşüyor, hiçbir hata/uyarı çıkmıyor. Bölüt ekrana sığmıyorsa okuma hızına göre (~17-20 karakter/sn) parçalara bölünüp bölüt süresine eşit pay ediliyor.

4.10 Bütçe ve ölçüm kapanışı

`ALTYAZI_HLS_GERIDE` → hls.js `liveSyncDurationCount` → izleyici canlı kenardan `liveSyncDurationCount × TARGETDURATION` kadar geride oynar — bu, altyazının **bütçesi**. İzleyici tarayıcısı dakikada bir gerçek gecikmesini (`Date.now()` - `playingDate()`) bildirir ve bu **ölçülmüş** değer varsayımın (`ALTYAZI_BUTCE_MS`) yerini alır. `SubtitleLagMetrics` her 60 sn'de bir `ALTYAZI_KAPSAMA` log satırı basar: tam/kısmi/hiç görünmeyen bölüt sayısı, ortalama/p50/p95/en kötü gecikme.

Ölçülen kapasite (CPU, 8 çekirdek, small, int8): çözümleme 0,80×, çeviri 4,39× (CPU'da ve seri), toplam 0,67×. Üretim gecikmesi p50 ~13 sn, p95 ~23 sn.

4.11 Video altyazısı — org.example.video.subtitle

Canlı boru hattıyla **paylaşılan** saf bileşenler (`SpeechSegmenter` , `SileroVad` , `TritonClient`); paylaşılmayan kısım worker'ın kendisi çünkü tek bir sınırlı dosya baştan sona işleniyor. **Zaman damgaları** `Instant.EPOCH` 'tan itibaren dosya-görelî — canlının mutlak duvar saatinin aksine, video için "izleyicinin

geride olduđu" diye bir kavram yok. Ayrı kuyruk (`VideoSubtitleQueue`) — video işleme I/O-ağırlıklı/saniyeler, altıyazı Triton/GPU-bağılı/dakikalar. Hata videonun kendisini etkilemez, **yeniden deneme yok** (tek seferlik best-effort) — Clip/VideoWorker'ın aksine.

4.12 Bilinen açık işler (CLAUDE.md'den)

1. **Yalnızca izlenen kanalı çözümle** — şu an kimse bakmasa da her kanal Whisper + 3 dil çeviriden geçiyor.
2. **Kuyruk kapasitesi küçültüldü ama .env'den ayarlanamıyor** (sabit 4).
3. **Çeviriyi GPU'ya al, kilidi kaldır** — kart ne kadar hızlı olursa olsun tavan ~4,4 kanal (CPU'da seri).
4. DVR/klip zinciri "yazıldı, DENENMEDİ" — `SegmentStreamTest` 'teki üç yeni kesme testi koşulmadı.
5. `ChannelDvrRecorder` 'a `-analyzeduration / -probesize` düzeltmesi uygulandı, gerçek etkisi **ölçülmedi**.

5. DVR / Geriye Sarma — org.example.dvr

Kanalların kesintisiz DVR kaydını, kaydın nesne depolamada (MinIO) tutulmasını, zaman çizelgesi hesaplamasını ve zaman aralığı çıkarımını (klip/geriye sarma) üstlenir.

Akış: `ChannelDvrRecorder` ffmpeg ile RTSP'den `-c copy` kopyalayıp `SegmentStream` üzerinden sabit süreli segmentlere böler, `DvrStorage` bunları MinIO'ya yazar ve `DvrRecorder` her segmenti `dvr_segments` tablosuna işler. `DvrArchive / DvrService` bu tablodan zaman çizelgesi üretir ve istenen aralığı segmentleri birleştirip ffmpeg ile keser (gerekirse boşlukları karanlık+sessiz dolguyla tamamlayarak).

ChannelDvrRecorder.java

```
ffmpeg -v warning -rtsp_transport tcp \
-analyzeduration 5000000 -probesize 10000000 \
-i rtsp://mediamtx:8554/<path> -c copy -f mpegts -
```

- `-c copy` : yeniden kodlama yok — kaynak kalitesinde kayıt.
- `-analyzeduration / -probesize` : varsayılan değerler bazı kaynaklarda "Could not find codec parameters" ile başlamadan düşüyordu.
- `-v warning` (error değil) : yeniden bağlanma/paket kaybı uyarı seviyesinde raporlanır, "error" ile stderr boş kalıp beklenmedik bitişin sebebi hiç görünmüyordu.
- **Geri baskı riski:** MinIO yavaşlarsa → yükleme yavaşlar → okuma durur → ffmpeg borusu dolar → bloke olur → RTSP tamponu taşıp canlı yayını etkileyebilir.
- Segment sıfır bayt geldiyse çizelgeye satır yazılmıyor.
- `kesSegmenti()` : DVR sinyali `KES` geldiğinde aktif segmenti erken kapatır — kayıt durmaz, yalnızca segment sınırı öne alınır.

SegmentStream.java

- Sürekli MPEG-TS akışını "başı sonu olan tek segment" gibi gösteren sarmalayıcı.
- **188 baytlık paket hizalaması:** kesim paket sınırında — ortadan kesilseydi çözücü senkronu bulana kadar veri atardı. Ölçüldü: parçalar birleştirildiğinde bayt bayt aynı dosya.
- **fMP4 değil TS:** fMP4 rastgele kesilemez, TS kendi kendini senkronluyor.
- Süre **duvar saatiyle** ölçülüyor (bayt sayısından çıkarılamaz, bit hızı değişken).

DvrStorage.java

- **Ayrı kova** (kliplerden): saklama süreleri zıt (segment 7 günde silinir, klip kalıcı) — ILM kuralları kova bazlı.
- **Saklama MinIO'nun kendisi tarafından (ILM)** uygulanıyor, süpürge kodu yok; veritabanı satırları ayrı temizlenmiyor (bilinen sınırlama).
- Çok parçalı yükleme (`PART_SIZE=10MB`) — düz `PUT Content-Length` istiyor; **ölçüldü:** chunked PUT'a MinIO HTTP 411 döndürüyor, ffmpeg bunu hata saymayıp geriye 0 baytlık nesne bırakıyordu.

DvrRecorder.java

- Kayıt işçilerini kanal yayın durumuyla eşitleyen orkestratör; yoklama (`dvr.sync-interval` , 10s) + Redis sinyaliyle anlık tetikleme.
- Kayıt kriteri: kanal yayında VE (`dvrEnabled` veya geçici manuel/planlı kayıt açık).
- MediaMTX'e ulaşılamazsa işçiler **kapatılmıyor** — kaydın deliği geri gelmez.

DvrSignal.java / DvrSignalEvent.java

- Redis pub/sub üzerinden `BASLAT / KES` sinyali. **Neden gerekli:** kaydediciyi yoklama aralığı (10 sn) kısa kayıtlarda ısıkalıyordu — 6 saniyelik kayıt, kaydedici daha başlamadan bitiyordu. Ayrılmış pub/sub istemcisi — aynı "41 saniye" tuzağı DVR'da da yaşanıp aynı şekilde düzeltilmiş.
- Kaybolursa yalnızca yavaşlar, doğruluk kaynağı hâlâ veritabanı.

DvrArchive.java

Aralık çıkarma zinciri:

```
MinIO segmentleri —birleştir→ ffmpeg -i pipe:0 -ss X -t Y -c copy → mp4
```

- `spans()` : `GAP_TOLERANCE=3sn` içindeki boşluklar birleştirilir.

- `sesAacMi()` : AAC ADTS→ASC bit akışı filtresi gerekip gerekmediğini tespit eder — **ilk satır** alınıyor çünkü fprobe birleştirilmiş TS'te aynı bilgiyi iki kez basabiliyor (`aac\naac`); tam eşitlik kontrolü bunu kaçırıp filtreyi eklemiyordu, sonuçta 1276 baytlık boş klip "HAZIR" işaretleniyordu (ölçüldü).

ses	filtre var	filtre yok
AAC	25,0 sn / 964 KB	0,13 sn — muxer duruyor
MP3	0 bayt — hiç başlamıyor	10,0 sn / 287 KB

- `spawn()` — aralık çıkarma komutu:

```
ffmpeg -v error -f mpegts -i pipe:0 -ss <offset> -t <süre> -c copy \
[-bsf:a aac_adtstoasc] \
-movflags frag_keyframe+empty_moov+default_base_moof -f mp4 pipe:1
```

Bayrak adı `default_base_moof` (`default_base_is_moof` DEĞİL) — yanlış ad ffmpeg'i başlatmadan düşürüyordu.

- **Boşluk doldurma (gap-fill, bu oturumda eklendi)**: segmentler arası boşluğa karanlık+sessiz dolgu:

```
ffmpeg -f lavfi -i color=c:black:s=WxH:r=FPS -f lavfi -i anullsrc=r=48000:cl=stereo \
-t < saniye> -c:v libx264 -preset ultrafast -pix_fmt yuv420p \
-c:a aac/mp2 -shortest -f mpegts -y <geçici.ts>
```

Fails-open tasarım: dolgu üretimi başarısız olursa boşluk eskisi gibi sessizce atlanır, klip yine üretilir.

DvrService.java

- `MAX_CLIP=2 saat` üst sınır.
- `clampToRecorded()` : istenen aralığı gerçekten kaydedilmiş bölüme kırpır — kullanıcı "kaydı başlat"a bastığı an ile kaydın gerçekten başladığı an aynı değil.

Veritabanı ve Config (DVR)

`dvr_segments` (V20): `channel_id` FK CASCADE, `nesne_anahtari` UNIQUE, indeks (`channel_id, basladi`).

Anahtar	Varsayılan	Not
<code>dvr.bucket</code>	dvr	Kliplerden ayrı kova.
<code>dvr.retention-days</code>	7	MinIO ILM ile uygulanır.
<code>dvr.segment-seconds</code>	30	Kısa → çok nesne, uzun → çökmede çok kayıp.
<code>dvr.recorder-enabled</code>	false	Yalnızca video-worker'da true.
<code>dvr.altiyazi-acik</code>	false	Altyapı hazır, üretimde hiç denenmedi.
<code>dvr.sync-interval</code>	10s	
<code>dvr.gap-fill-enabled</code>	true	
<code>dvr.gap-fill-min-seconds</code>	1	
<code>dvr.gap-fill-timeout-seconds</code>	20	

6. Klip, Ekran Görüntüsü ve Depolama

6.1 org.example.clip

Kanaldan zaman aralığı seçerek ya da manuel/planlı kayıtlı klip çıkarma, bu işlerin veritabanı destekli kuyruğu, MinIO'ya yazımı, önizleme/kapak üretimi ve indirme/izleme adreslerinin imzalanması.

ChannelRecordingGate.java

- Kanalın DVR'ı kapalıyken manuel/planlı kayıt süresince geçici olarak kaydı açan/kapatan kapı.
- **Mimari geçmiş**: Eskiden `mediaMtx.applyPath(record=true)` ile path yeniden başlatılıyordu — **tüm izleyiciler için canlı yayını kısa süre kesiyordu**. Artık veritabanı sinyali + `DvrSignalEvent` kullanılıyor, yayına hiç dokunulmuyor.

ClipQueue.java / ClipConsumer.java

- Redis liste tabanlı kuyruk — **Redis doğruluk kaynağı değil, bildirim kanalı**; kalıcı durum `clips` tablosunda. `BLMOVE` ile atomik taşıma (basit `BRPOP` olsaydı işçi işi aldıktan hemen sonra çökerse iz bırakmadan kaybolurdu).
- İki tetikleyici: Redis bildirimi (hızlı) + veritabanı süpürücüsü (`sweep`, güvenlik ağı).

ClipService.java

- **Süre üst sınırı** (`clips.max-duration-minutes`, 120 dk).
- **Geriye sarma koşulu ve tam kapsama kontrolü yalnızca ARALIK kaynağında** aranır — manuel/planlı kayıtlı DVR kapalı olabilir (geçici açılmış).

ClipStorage.java

- Klipler nesne depolamasına yazılır çünkü indirme backend'den geçmemeli (2 saatlik klip 6 Mbps'te ~5,4 GB).
- İki ayrı MinioClient**: iç ağ (`internalReadUrl`, ffmpeg için) ve dış/public (`streamUrl` / `downloadUrl`, tarayıcı için) — S3 v4 imzası Host başlığını da imzaladığı için sonradan host değiştirmek imzayı geçersiz kılar.

ClipWorker.java

- `claim()`: `atCapacity()` kontrolü (`clips.concurrency=2`) — sınırsız olsaydı canlı yayın etkilenirdi.
- `process()`: `dvrService.extractStream()` ile **doğrudan** `InputStream` alınır (`readEntity()` çalışmıyor — "Request could not be mapped to type InputStream" hatası yaşanmış).
- dvrBekle**: segment kapanmadan çizelgeye satır yazılmadığı için, kaydı durduran kullanıcı segment henüz açıkken soruyor olabilir; her turda yeni transaction açılır (Hibernate ön belleği yüzünden).
- altyaziUret**: klip aralığındaki canlı altyazıyı WebVTT'ye çevirip sidecar olarak yükler; hata klip üretimini durdurmaz.
- onizlemeUret**: `MediaTools/VideoEncoder` (donanım hızlandırma) **kullanılmıyor** çünkü `Clipworker` backend konteynerinde (donanım kodlayıcı yok); sabit yazılım (libx264) kullanılıyor.

```
ffmpeg -t <uzunluk> -i <internalReadUrl> -an -vf scale=<width>:-2 \
-c:v libx264 -preset veryfast -crf 30 -pix_fmt yuv420p \
-movflags +faststart -y <geçici.mp4>
```

Kapak karesi önizleme klibinden çıkarılıyor (kaynaktan değil — ikinci MinIO okuması gerekmiyor).

RecordingService.java

- `stop()`: **kayıt satırının silinmesi ile klip üretimi ayrı transaction'larda** — aynı işlemde olsalardı klip doğrulaması başarısız olunca rollback tetiklenir, satır geri gelir ve kullanıcı kaydı **bir daha hiç durduramazdı** (gerçekten yaşanmış).

ScheduledRecordingScheduler.java

- 30 sn'de bir tik. `ON_PAY=15s`: kayıt aralığının başından 15 sn önce açılıyor çünkü MediaMTX segment sınırında başlatıyor.

6.2 org.example.screenshot

Canlı yayından tek kare yakalama. Kliplerden farkı: **kare tarayıcıda yakalanır**, sunucu tarafı yakalama/işleme yok.

Not — dokümantasyon tutarsızlığı: V14 migration yorumu "kareyi SUNUCU yakalıyor" der; gerçek kod tarayıcıda yakalayıp yüklediğini söylüyor ve gerekçelendiriyor (HLS gecikmesi 6-20 sn, ffmpeg yalnızca işçi konteynerinde). Migration yorumu muhtemelen eski tasarımdan kalma.

- `ScreenshotService.capture()`: anahtar **sunucuda** üretilir; görsel **backend üzerinden geçer** (klip/video'nun aksine — birkaç yüz KB için imzalı adres dansı gereksiz).
- Ayrı MinIO kovası** — galerinin kendi saklama politikası var.

6.3 org.example.storage

QuotaService.java

- Ayrı sayaç tablosu yok** — boyutlar zaten `clips.size_bytes` / `screenshots.size_bytes` / `videos.size_bytes` sütunlarında, toplam sorguyla alınıyor (ikinci bir doğruluk kaynağı er geç kayardı).
- Kota dolunca yeni iş reddedilir, var olan silinmez.**

RetentionSweeper.java

- `@Scheduled every={storage.sweep-interval}` (1h). **Varsayılan olarak kullanıcı verisi silinmiyor** (`storage.clip-retention=0`) — "Baskıyı kota kursun, saat değil."
- İstisna**: başarısız klipler varsayılan 7 gün sonra silinir (dosyaları zaten yok).

StoragePaths.java

- MinIO klasör düzeni: `<kullanıcı>/<kanal>/<id>.mp4`. **Kullanıcı en üstte** çünkü içerik kişisel.
- `slug()`: Türkçe karakterler elle eşlenip NFD normalize + diacritic temizleme.

Veritabanı ve Config (Klip/Ekran Görüntüsü/Depolama)

Migration	İçerik
V5	<code>clips</code> tablosu (kuyruk + durum), kısmi indeks <code>idx_clips_bekleyen</code> .
V13	<code>clips.origin</code> (ARALIK/MANUEL_KAYIT), <code>active_recordings</code> .
V14	<code>screenshots</code> .
V15	<code>planli_kayitlar</code> .
V18/V21	<code>clips.channel_id</code> CASCADE → SET NULL ince ayarı + <code>channel_name</code> kopyası.
V29/V30	<code>clips.preview_key</code> / <code>thumbnail_key</code> .

Önemli config: `clips.max-duration-minutes` (120), `clips.concurrency` (2), `clips.dvr-bekleme` (45s), `storage.user-quota-bytes` (20 GB), `storage.failed-clip-retention` (P7D).

7. Video Kütüphanesi ve Radyo

7.1 org.example.video

Video kütüphanesi — yükleme (imzalı adresle iki adımlı), işleme (ffmpeg ile metadata, faststart, küçük resim + önizleme), CRUD ve otomatik altıyazı. Klip modülüyle paralel birçok desen paylaşır (tablo=kuyruk, SKIP LOCKED, Redis bildirim/DB süpürücü çifti).

entity.Video

- `objectKey` sunucu üretir, istemciden alınmaz. `thumbnailKey` / `previewKey` / `thumbnailAtSeconds` / `thumbnailIsUpload`: üç farklı küçük-resim durumu (otomatik/kullanıcı seçimi/kullanıcı yüklemesi).

VideoStatus / VideoSubtitleStatus

- Video: `YUKLENIYOR` → `ISLENIYOR` → `HAZIR/HATA` — klipten farkı **iki** bekleme durumu (tarayıcı hâlâ mı yüklüyor, yoksa yarım mı kaldı).
- Altıyazı durumu video kuyruğundan **ayrı** — STT dakikalar sürebilir, aynı havuzda thumbnail işini yavaşlatırdı.

MediaTools.java

- Kabuk kullanmadan** argüman listesiyle çalıştırılır (imzalı URL'deki `&/?` yorumlanmaz).
- `needsFastStart()`: MP4 kutu yapısını elle ayrıştırıp `moov` mü `mdat` mı önce geliyor bakar — `moov` sonda ise tarayıcı oynatmaya başlamadan tüm dosyayı indirmeye çalışır.
- Küçük resimde yalnızca **çözme** hızlandırılır (mjpeg kodlayıcı yazılımda).

VideoStorage.java

- Klip kovalarından **ayrı** kova (video TTL uzun, klip TTL kısa — aynı olsaydı video ortasında kopardı).
- `uploadUrl`: Content-Type imzaya **dahil edilmiyor** — dahil edilseydi en ufak header farkında MinIO 403 dönerdi.

VideoService.java

- Yükleme **iki adımlı**: `startUpload` (imzalı PUT) → `completeUpload` (MinIO'da gerçekten var mı doğrulanır → `ISLENIYOR`).
- `reconcileStaleUploads`: nesne MinIO'da varsa bildirim kaybolmuş (işleme alınır), yoksa yükleme gerçekten yarım kalmış (`HATA`).

VideoWorker.java

- `process`: probe (video akışı yoksa reddet) → gerekiyorsa faststart remux → küçük resim → önizleme (hata ölümcül değil) → `markReady`.
- Yarış durumu**: iş başlarken kullanıcı görseli yoktu ama işleme sırasında yüklendiyse kullanıcının seçimi kazanır, işçinin ürettiği kare silinir.
- Video oynatılabilir hale geldikten sonra `subtitleEnabled && hasAudio` ise altıyazı kuyruğuna girer.

video.subtitle alt paketi

- Bkz. §4.11 — canlı boru hattıyla paylaşılan bileşenler (`SpeechSegmenter`, `SileroVad`, `TritonClient`), zaman damgaları dosya-göreliliği (mutlak değil).

7.2 org.example.radio

Radyo yayınlarının CRUD'u — kanallarla aynı altyapıyı (MediaMTX) paylaşan ama daha basit bir model (rendition/DVR yok).

RadioSourceKind

- DOGRUDAN**: adres doğrudan MediaMTX'e (HLS/RTSP/RTMP/SRT).
- KOPRU**: MediaMTX okuyamadığında (Icecast/Shoutcast düz MP3) ffmpeg araya girip AAC'ye kodluyor — ölçülen maliyet %2.6 CPU/radyo. Kullanıcıdan **açıkça alınır, tahmin edilmez** — yanlış tahminin cezası görünür hata değil, hiç başlamayan bir yayın.

AudioBridgeCommand.java

- KOPRU modu için ffmpeg komut satırı; **kabukta** çalıştığı için iki katmanlı güvenlik: `SAFE_URL` regex (boşluk/tırnak/ ; / | \$ yasak) + izinli şema kümesi.
- `-vn`: Icecast albüm kapağını video akışı sanıp düşmesin diye.

RadioService.java

- `requirePathFree()` hem `Radio` hem `Channel` tablosuna bakar — MediaMTX'te **aynı isim alanını paylaşıyorlar**.
- `capacity()`: radyolar `channels.max-active` 'e **dahil değil** — farklı ölçek (radyo %2.6 CPU vs video rendition %14).

Veritabanı ve Config (Video/Radyo)

`videos` (V9): kısıtlar `videos_durum_gecerli`, `videos_hazir_eksiksiz` (HAZIR video mutlaka `thumbnail_key+size_bytes`'a sahip olmalı). V28: `subtitle_status` / `subtitle_langs` / `subtitle_error`. `radios` (V8): kısıt `radios_kaynak_turu_gecerli`.

Önemli config: `videos.max-upload-bytes` (5 GB), `videos.encoder` (NVENC), `videos.subtitle-enabled` (false), `videos.subtitle-concurrency` (1 — Triton kendi içi eşzamanlılığı zaten yönetiyor), `radios.max-active` (32 — ölçülen: 32 radyo ≈ 0,8 çekirdek).

8. Denetim İzi, Analitik ve Sistem Logları

8.1 org.example.etkinlik

Kullanıcı davranışı denetim izi (giriş/çıkış, izleme/dinleme, admin/içerik eylemleri) + admin panel analitik dashboard'unun beş uç noktası. Tek tablo (`etkinlik_kayitlari`), `EtkinlikTuru` enum'ı, JSON `detay` alanı.

EtkinlikService.java

- Cross-cutting annotation/interceptor yerine çağrı noktalarında açık tek satırlık çağrılar — ~10 çağrı noktası için yeni soyutlama kurmaya değmez.
- `kaydet`: kullanıcı bulunamazsa (eşzamanlı silinme) `null` kullanıcild ile sessizce kaydeder — "denetim izi bir istek akışını asla durdurmamalı."

AnalitikService.java

- Kritik kural:** hiçbir alan sahte/tahmini sayı üretmez — enstrümantasyonu olmayan veriler `null` döner, frontend "ölçülüyor" gösterir.
- `anlikTrafikMbps()`: MediaMTX'in **kümülatif** `bytesReceived` sayacından iki örnekleme arası farkı hesaplar; ilk çağrıda `null` (uydurma ilk değer yerine). Negatif fark (MediaMTX yeniden başlamış) da `null`'a düşer.
- `kullaniciAktivitesi()`: `sum((detay->'sureMs')::bigint)` — Postgres `sum(bigint)` numeric döndüğü için doğrudan `(Long)` cast `ClassCastException` atardı, `Number.longValue()` ile alınıyor.
- `genelBakis()`: yedi bileşenin sağlık durumu, her biri kendi try/catch'inde — biri çökerse diğerleri etkilenmez. `yayinSagligi()` ve `canliDurum().yayindakiKanal` **aynı hesabı paylaşır** — iki yerde iki kez sayılıp tutarsızlık çıkmasın diye.

PrometheusClient.java

- `/api/v1/query` instant-query. Hata durumunda sessizce `null`/boş harita.

8.2 org.example.sistemlog

Konteyner loglarını Loki üzerinden çekip Türkçe/kullanıcı dostu mesajlara çeviren küçük (4 sınıf) paket.

LokiClient.java

- Backend **docker.sock** 'a hiç dokunmaz — yalnızca Loki'nin HTTP API'sine sorgu atar (Promtail dokunuyor).

SistemLogYorumlayici.java

- Saf, DI'sız, sıralı kural motoru (ilk eşleşen kazanır). Örnek kurallar:

Servis	Regex	Seviye	Şablon
triton	<code>...CUDA...out of memory</code>	HATA	"{model} GPU belleği dolduğu için isteği işleyemedi."
video-worker	<code>ALTYAZI (\S+) \[.*?\] (\S+) →</code>	BASARI	"{kanal} için altyazı üretildi."
backend	<code>No handler waiting for message: \[unsubscribe</code>	BILGI	"Altyazı aboneliği henüz açılmadı (normal)."
mediamtx	<code>is not configured</code>	HATA	"Tanımsız bir path'e istek geldi."

- Kural eşleşmezse: JSON yapılı loglarda kendi `level` alanına bakılır (ERROR/WARN gösterilir); JSON olmayanlarda yalnızca `error` / `fatal` / `exception` / `panic` geçener HATA sayılır, gerisi süzülür.
- `CEKME_ORANI=5`: ham çekim istenen limitin 5 katı — rutin gürültünün büyük kısmı yorumlama aşamasında elendiği için.

Veritabanı ve Config

`etkinlik_kayitlari` (V23, V25): indeksler `(kullanici_id, olusturma_zamani desc)`, `(tur, olusturma_zamani desc)`. Sistemlog: veritabanı bağımlılığı yok, tek config `loki.url`.

9. Veritabanı Şeması — Tüm Migrasyonlar

#	Dosya	Konu
V1	users_and_roles	Kullanıcı/rol temel şeması
V2	turkce_rol_isimleri	Rol isimleri Türkçe (ASCII)
V3	rol_isimleri_aksanlı	Rol isimleri aksanlı
V4	channels	Kanal tablosu
V5	dvr_ve_klipler	Klip + <code>dvr_enabled</code>
V6	kanal_renditionlari	Kanal bazlı çözünürlük merdiveni
V7	dvr_rendition	—
V8	radyolar	Radyo tablosu
V9	videolar	Video tablosu
V10	video_kucuk_resim_yukleme	Kullanıcı küçük resim yüklemesi
V11	video_onizleme_klibi	Video önizleme klibi
V12	kanal_kaynak_cozunurlugu	Master playlist tuzağı çözümü
V13	manuel_kayit	Manuel kayıt + klip origin
V14	ekran_goruntuleri	Ekran görüntüsü tablosu
V15	planli_kayitlar	Planlı kayıt emirleri
V16	manuel_kayit_dvr_bizden	DVR-bizden bayrağı
V17	kayit_kaynak_kalitesinde	—
V18	klip_kanal_cascade	Klip-kanal CASCADE (geçici)
V19	altyazilar	Altyazı tablosu
V20	dvr_segmentleri	DVR segment tablosu
V21	kanal_silmede_icerik_korunabilsin	CASCADE -- SET NULL ince ayarı
V22	video_izlenme	Video view count
V23	etkinlik_kayitlari	Denetim izi tablosu
V24	klip_izlenme	Klip view count
V25	etkinlik_tur_hedef_indeksi	Denetim izi ek indeksler
V26	altyazi_baslangic_indeksi	Altyazı performans indeksi
V27	klip_altyazi_dilleri	Klip altyazı dilleri
V28	video_altyazi	Video altyazı durumu
V29	klip_onizleme_klibi	Klip önizleme klibi
V30	klip_kapak_resmi	Klip kapak resmi

10. Frontend — Genel Mimari

Ana teknolojiler (`package.json`):

Kategori	Kütüphane
Çekirdek	React ^19.2.7
Routing	react-router-dom ^7.18.2
Derleme	Vite ^8.1.1, TypeScript ~6.0.2
Stil	Tailwind CSS v4 (<code>@tailwindcss/vite</code>)
UI kit	Radix UI primitifleri + <code>class-variance-authority</code> — shadcn/ui deseni
HLS oynatma	<code>hls.js</code> ^1.6.16
Bildirim	<code>sonner</code>
Derleyici	<code>babel-plugin-react-compiler</code> (otomatik memoization)

Ayrı bir global state kütüphanesi (Redux/Zustand) **yok** — React context (`AuthContext` , `PlayerContext`) ve bileşen içi state kullanılıyor.

Giriş noktası (`main.tsx`): Uygulama render edilmeden önce `ayarlariYukle()` çağrılıp beklenir — oynatıcı ayarları (`ALTYAZI_HLS_GERIDE` gibi) hls.js kurulurken okunuyor ve kurulum yalnızca bir kez oluyor; sonradan gelirse ilk oynatıcı yanlış değerle kurulup canlı yayında görünür bir kesinti gerektirirdi.

Routing (App.tsx): İki ayrı kabuk — normal kullanıcı sayfaları `AppLayout` (mozaik oynatıcılar, sol çubuk), yönetim sayfaları tamamen ayrı `AdminLayout` içinde (izleme uygulamasının arayüz mantığıyla karışmasın diye).

Path	Sayfa	Koruma
<code>/giris</code>	LoginPage	Açık
<code>/izle</code>	WatchPage	Giriş yapmış herkes
<code>/kanallar</code> , <code>/radyolar</code> , <code>/geriye-sarma</code> , <code>/klipler</code> , <code>/videolar</code> , <code>/galeri</code> , <code>/profil</code>	İlgili sayfalar	Giriş yapmış herkes
<code>/yonetim/*</code>	Admin sayfaları	Yalnızca Yönetici

10.1 Kimlik Doğrulama (src/auth/)

Frontend Keycloak'a doğrudan yönlendirme yapmıyor — kullanıcı adı/şifre backend'in `/api/auth/login`'ine POST edilip Keycloak arkada backend tarafından sorgulanıyor.

- Token'lar** `sessionStorage`'da tutuluyor, `localStorage` değil — paylaşılan bir makinede bir sonraki kullanıcıya açık oturum devretmesin diye.
- `decodeClaims()`: JWT payload'ı **imza doğrulanmadan** çözülüyor — yalnızca arayüzü şekillendirmek için (hangi menü görünsün), gerçek yetki kararı her istekte backend'de.
- Token yenileme: **tek uçuşlu kilit** (`refreshInFlight`) — eşzamanlı isteklerin aynı anda ayrı yenileme denemesi Keycloak'ın kullanılmış refresh token'ı reddedip oturumu düşürmesine yol açardı.

10.2 API Katmanı (src/api/)

- Tüm istekler göreceli path (`/api/...`) — Vite proxy CORS'u ortadan kaldırıyor.
- `upload.ts`: büyük dosya yüklemeleri `fetch` değil `XMLHttpRequest` ile — `fetch` yükleme ilerlemesi bildirmiyor. `Authorization` header'ı **bilinçli eklenmiyor** — hedef MinIO, header eklenirse imzalı URL'i reddediyor.
- `channelsApi.remove` **POST** (DELETE değil) — şifre gövdede.
- `clipsApi.links` JSON döner, yönlendirme değil — CORS `Location` başlığını okuyamıyor.
- `dvrApi.streamUrl` token gerektirdiği için `fetch` ile alınıp blob'a çevriliyor.

10.3 Yardımcılar (src/lib/)

- `sekmekimligi.ts`: Her sekmeye özgü kalıcı kimlik. **Gerçek prod hatası:** `crypto.randomUUID` yalnızca "secure context"te (HTTPS/localhost) mevcut; düz HTTP + LAN IP ile açıldığında (`http://192.168.1.200:3000`) tanımsız kalıp sayfayı çökertiyordu — ölçülmüş/gözlemlenmiş. Çözüm: varlık kontrolü + `Math.random` tabanlı fallback.
- `renditions.ts`: MediaMTX rendition'ları tek master playlist'te birleştirmedeği için kalite seçimi arayüzde elle sunuluyor.

11. Frontend — Layout, Rehberli Tur ve Oynatıcı

11.1 Layout Bileşenleri

AppLayout.tsx: Sidebar daraltma/genişletme `localStorage` ile kalıcı (`w-16 ↔ w-60`). Aktif nav öğesi tam genişlikte kapsül (`rounded-full`). Profil bloğu listeden ayrı ("içerik sekmesi değil, hesaplara ilgili"). `PersistentPlayers` / `PersistentRadio` bilerek `<Outlet>`'in **dışında** — sayfa değişince unmount olup yayının kesilmemesi için.

AdminLayout.tsx: İzleme uygulamasından **tamamen ayrı** kabuk — eskiden Kullanıcılar/Etkinlikler/Analitik izleme uygulamasının sol çubuğunda sıradan sayfalarla ama yönetici olmayanlar için işlevsizken izleme arayüzünün sürekli arka planda yaşamasına sebep oluyordu. Görsel dil bilinçli olarak AppLayout ile aynı tutulmuş.

11.2 Rehberli Tur Sistemi (src/components/tour/)

Küçük, kütüphanesiz, elle yazılmış tur motoru — hazır çözümler (Joyride/Shepherd) React 19 uyumu belirsiz bağımlılık getirirdi.

- Vurgu tekniği: `box-shadow: 0 0 0 9999px rgba(0,0,0,.72)` — dört ayrı örtü yerine tek saydam kutu (piksel kayması olmasın diye).
- Hedefler `data-tour="..."` veri özniteliğiyle işaretleniyor, CSS sınıfı değil — Tailwind sınıfları sürekli değişiyor, bir sınıfı düzenleyen kişi turu bozduğunu fark edemezdi.
- Gösterilecek adımlar açılışta bir kez hesaplanıp donduruluyor — tur sırasında yeni bir öğe açılırsa dizi uzayıp indeks kayması olmasın diye.

11.3 Oynatıcı (src/player/ ve HlsPlayer.tsx)

HlsPlayer.tsx: `capLevelToPlayerSize: true` — 4x4 gridde her karo ~480px olduğundan bu olmadan hls.js her karoda 1080p çözerdi. `lowLatencyMode: true` + `liveSyncDurationCount: hlsGerideOku()` — **ALTYAZI_HLS_GERIDE ile birebir aynı mekanizma** (bkz. §4.10). Otomatik oynatma: önce sesli dener, reddedilirse sessize alıp tekrar dener (ölçülmüş: "Autoplay is only allowed..." LAN IP'de ilk açılışta).

PersistentPlayers.tsx: Tek ağaç kuralı — görünüm (tam/mini/büyük) yalnızca CSS sınıflarıyla değişir, bileşen ağacı hiç değişmez (farklı JSX dalları React'ı söker, her geçişte yayın baştan bağlanırdı). **Geri sarma bug'ı (17 Ağustos):** rewind modunda `HlsPlayer` unmount olurken kendi `captureRef.current = null` yazıyordu — kare yakalama "Görüntü henüz hazır değil" veriyordu; çözüm ayrı bir `tileCapture` nesnesi.

oplayerAyarlari.ts: Backend'den gelen ayarlar **modül düzeyi** değişken (React state değil) — HlsPlayer ayarı hls.js kurulurken okur, kurulum bir kez olur; state'ten gelseydi yeniden kurulum (görünür kesinti) gerekirdi. Sıfır/negatif **hlsGeride** reddediliyor — hls.js kullanıcı ayarı verilmemiş sayıp sunucunun **PART-HOLD-BACK=0.5** önerisine düşerdi.

usePresence.ts: **HEARTBEAT_MS=15000** + **pagehide** 'da **keepalive:true** ile DELETE — MediaMTX'in reader sayısı yeniden bağlanmalarda çok sayıyor.

12. Frontend — Kullanıcı Sayfaları

WatchPage.tsx (**/izle**): Kasıtlı olarak boş (**return null**) — gerçek içerik **PersistentPlayers** 'ta, route'un dışında yaşıyor.

ChannelsPage.tsx: **streaming === null** ("Bilinmiyor") ile **!streaming** ("Akmıyor") ayrımı — MediaMTX'e ulaşamaması ile gerçek arıza karıştırılmasın diye.

VideosPage/ClipsPage: YouTube tarzı arayüz. Hover önizleme 400ms gecikmeli (ızgaranın üzerinden geçmek onlarca video indirmeye başlatmasın diye). **crossOrigin="anonymous"** **ŞART** — olmadan **<track>** isteği "no-cors" ile opak sayılıp alt yazı sessizce yüklenmiyor. İndirme gizli **<a>** elemanı ile — token gerektirdiği için **<a href>** doğrudan çalışmıyor.

GaleriPage: Büyütme katmanı düz **div**, Radix Dialog değil ("tek görsel için odak tuzağı gereksiz ağırlık"). Ok tuşlarıyla komşu kareye geçiş.

RadiosPage: Liste state'i **PlayerContext** 'e taşınmış — istasyon değişince dinleyici sayısının anında güncellenmesi için paylaşılan tek state gerekiyor.

DvrPage.tsx — en karmaşık sayfa:

- PREVIEW_MAX_SECONDS=30dk** — segment tamamen belleğe indirildiği için (2 saatlik seçim 6 Mbps'te ~5 GB, tarayıcıyı düşürdü).
- FILMSTRIP_COUNT=6** — "6 kare ~2,5 MB indiriyor, daha fazlası hem sunucuya hem beklemeye yansır."
- Timeline.tsx:** yalnızca kayıt bulunan bölgelerden seçim başlatılabilir. Ölçekleme piksel değil **oran** üzerinden. 2 saniyeden kısa sürüklemeye tıklama sayılıyor.
- Önizlemeden kare yakalama canlı yakalamadan **daha kesin:** **capturedAt = previewRange.start + video.currentTime** — "şimdi" kullanılmıyor, HLS gecikmesi tahmini gerekmiyor.
- Alt yazı overlay'i DVR önizlemesinde de **aynı mekanizmayla** çalışıyor — mutlak zaman damgası eşleşmesi burada da geçerli, yalnızca **playingDate()** **previewRange** 'e göre hesaplanıyor. Canlı ile DVR arasında kod paylaşımını gösteren önemli bir mimari detay.

ProfilePage: **QuotaCard** — kota dolunca yeni iş reddediliyor ama var olan içerik silinmiyor, "ne silineceğine kullanıcı karar veriyor."

Alt bileşenler: **DeleteChannelDialog** — şifre + üç bağımsız bayrak (klip/ekran görüntüsü/DVR ayrı seçilebiliyor). **VideoUploadDialog** — üç aşamalı doğrudan-depolama akışı (kayıt aç → MinIO'ya PUT → backend'e doğrula); üçüncü adım başarısız olsa bile bir süpürücü aynı işi yapıyor. **RadioFormDialog** — kaynak türü kart olarak seçiliyor ("yanlış seçimin cezası görünür hata değil, hiç başlamayan bir yayın").

13. Frontend — Admin Sayfaları

AdminGenelBakisPage.tsx: Üç paralel API çağırısı, 15 sn'de bir otomatik yenileme. Kritik ayrım: "Sistem Sağlığı" (erişilebilirlik, yeşil/kırmızı) ile "Servis Metrikleri" (Prometheus'tan gerçek sayı) ayrı kart grupları — **Olcum** bileşeni ölçülmeyen alanı sessizce sıfır değil italik "ölçülüyor" gösteriyor.

AdminAnalitikPage.tsx: Beş modül, yedi paralel endpoint. "En yoğun saat" istemci tarafında türetiliyor (backend doğrudan döndürmüyor).

AdminEtkinliklerPage.tsx: 23 etkinlik türü, her biri Türkçe etiket + rozet rengi. CSV export mevcut filtreyle eşleşen **tüm** kayıtları (yalnızca ekrandaki sayfayı değil) 200'lük parçalar halinde döngüyle topluyor; UTF-8 BOM ekleniyor (Excel Türkçe karakter bozulmasını diye).

AdminUsersPage.tsx: Rol değişimi ayrı "kaydet" düğmesi olmadan Select seçilir seçilmez uygulanıyor. "Eşitle" Keycloak'taki değişiklikleri yerel tabloya yansıtıyor, "orphaned" (Keycloak'ta artık olmayan) kayıtları raporluyor ama silmiyor.

AdminSistemLoglarPage.tsx: **Sayfalama tuzağı** — Loki SQL tablosu olmadığı için sunucudan "sayfa N'i ver" istenemiyor; en fazla 500 kayıt çekilip **istemci tarafında** dilimleniyor, **Sayfalama** 'ya geçilen **total** gerçek sistem geneli sayı değil.

Paylaşılan bileşenler: **Sayfalama.tsx** (akıllı sayfa numarası gösterimi, ellipsis-windowing), **Olcum / StatKart / BolumBasligi** (projenin "ölçmeden sayı verme" ilkesinin UI karşılığı).

Ek: İncelenen Kaynak Dosyalar

Bu doküman, repo kökü **/home/fb/IdeaProjects/Yayin_Platformu** altındaki şu ana klasörlerin tam taranmasıyla üretildi:

- src/main/java/org/example/{auth,user,viewer,channel,media,playback,VAD,subtitle,dvr,clip,screenshot,storage,video,radio,etkinl**
- src/main/resources/db/migration/V1..V30**
- src/main/resources/application.properties**
- triton/** (export_models.py, model_repository/whisper/1/model.py, templates/marian_model.py)
- frontend/yayin-frontend/src/{api,auth,lib,components,player,pages}/**